

Lecture 3. Linear Regression

COMP90051 Statistical Machine Learning

Lecturer: Jean Honorio



This lecture

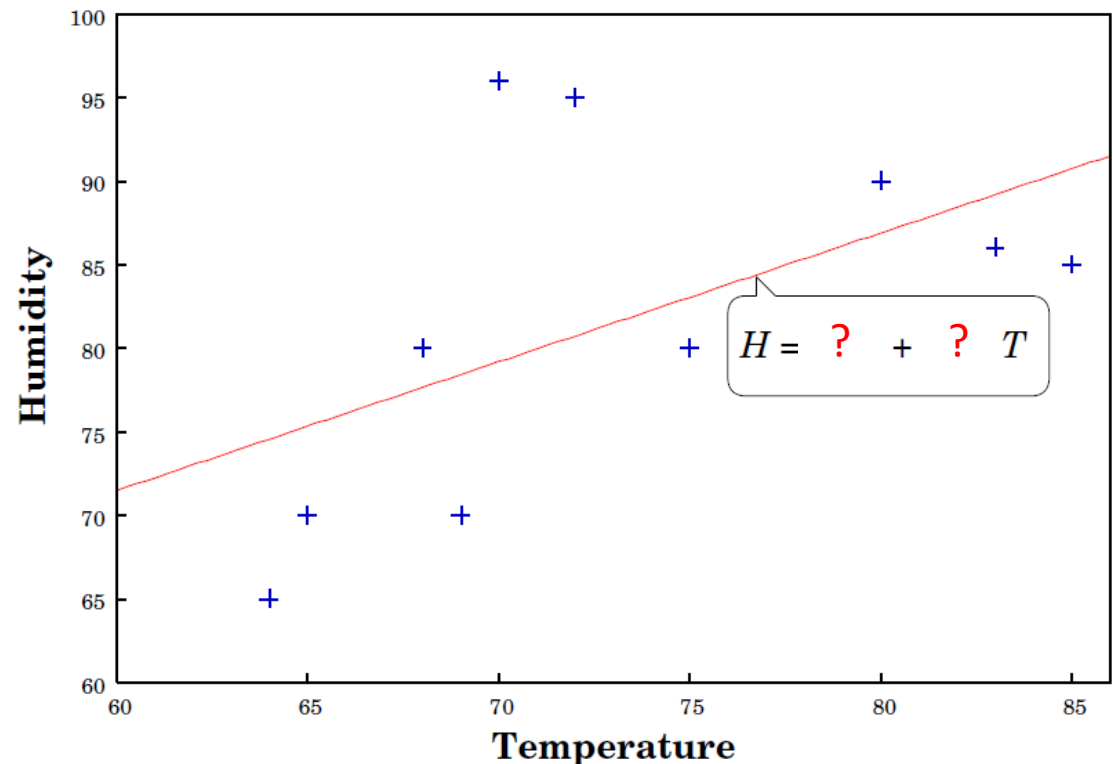
- Linear regression
 - * Simple model (convenient maths at expense of flexibility)
 - * Often needs less data, “interpretable”, lifts to non-linear
 - * Derivable under all Statistical Schools
 - This week: Frequentist + Decision theory derivations
 -  Later in semester: Bayesian approach
 - * Convenient optimisation: Training by “analytic” (exact) solution
- Basis expansion: Data transform for more expressive models

Linear Regression via Decision Theory

A warm-up example

Example: Predict humidity from temperature

Temperature	Humidity
TRAINING DATA	
85	85
80	90
83	86
70	96
68	80
65	70
64	65
72	95
69	70
75	80
TEST DATA	
75	70



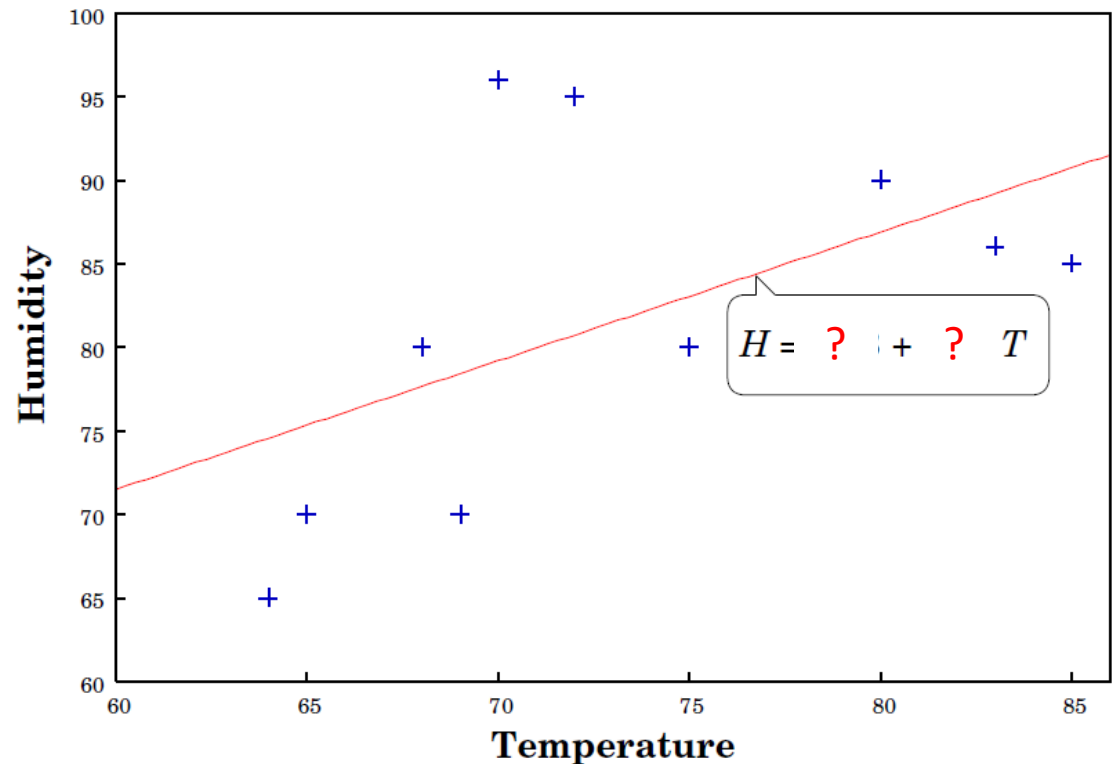
In regression, the task is to predict numeric response (*aka* dependent variable) from features (*aka* predictors or independent variables)

Assume a linear relation: $H = a + bT$

(H – humidity; T – temperature; a, b – parameters)

Example: Problem statement

- The model is
 $H = a + bT$
- Fitting the model = finding “best” a, b values for data at hand
- Important criterion: minimise the **sum of squared errors** (aka residual sum of squares)



Example: Minimise Sum Squared Errors

To find a, b that minimise $L = \sum_{i=1}^{10} (H_i - (a + b T_i))^2$

set derivatives to zero:

$$\frac{\partial L}{\partial a} = -2 \sum_{i=1}^{10} (H_i - a - b T_i) = 0$$

if we know b , then $\hat{a} = \frac{1}{10} \sum_{i=1}^{10} (H_i - b T_i)$

$$\frac{\partial L}{\partial b} = -2 \sum_{i=1}^{10} (H_i - a - b T_i) T_i = 0$$

if we know a , then $\hat{b} = \frac{1}{\sum_{i=1}^{10} T_i^2} \sum_{i=1}^{10} (H_i - a) T_i$

High-school optimisation:

- Write derivative
- Set to zero
- Solve for model
- (Check 2nd derivatives)

Example: Analytic solution

- We have two equations and two unknowns a, b
- Rewrite as a system of linear equations

$$\begin{pmatrix} 10 & \sum_{i=1}^{10} T_i \\ \sum_{i=1}^{10} T_i & \sum_{i=1}^{10} T_i^2 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^{10} H_i \\ \sum_{i=1}^{10} T_i H_i \end{pmatrix}$$

- **Analytic solution:** $a = 25.3, b = 0.77$
- (Solve using `numpy.linalg.solve`)

More general decision rule

- Adopt a linear relationship between response $y \in \mathbb{R}$ and an instance with features $x_1, \dots, x_m \in \mathbb{R}$

$$\hat{y} = w_0 + \sum_{i=1}^m x_i w_i$$

Here $w_0, \dots, w_m \in \mathbb{R}$ denote weights (model parameters)

- Trick:** add a dummy feature $x_0 = 1$ and use vector notation

$$\hat{y} = \sum_{i=0}^m x_i w_i = \mathbf{x}' \mathbf{w}$$

Mini Summary

- Linear regression
 - * Simple, effective, “interpretable”, basis for many approaches
 - * Decision-theoretic frequentist derivation

Next:

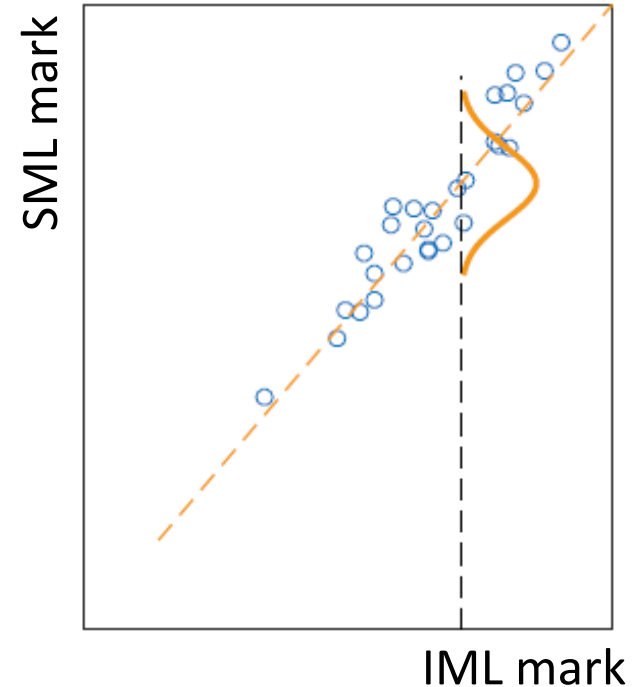
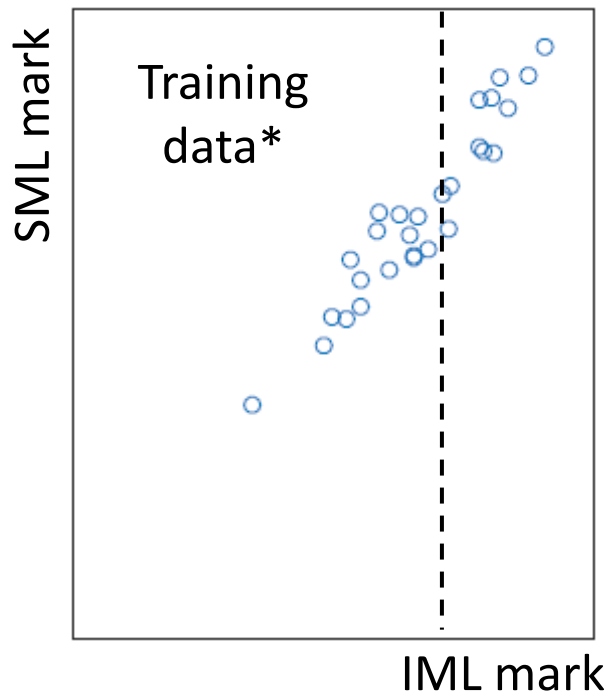
Frequentist derivation; Solution/training approach

Linear Regression via Frequentist Probabilistic Model

Max-Likelihood Estimation

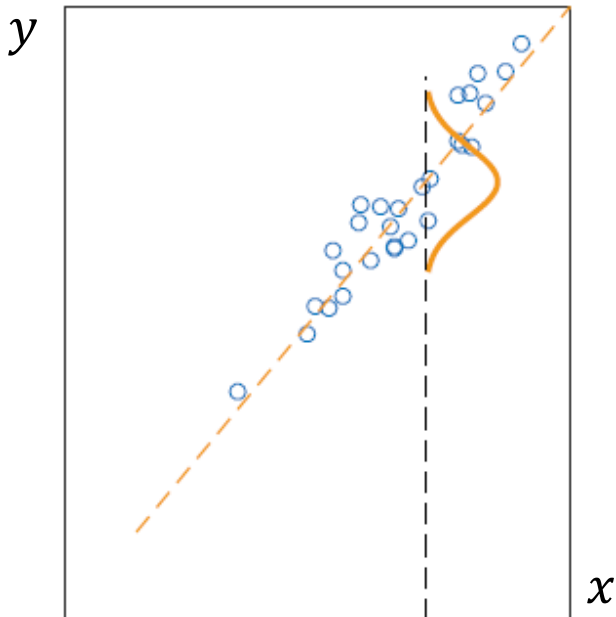
Data is noisy!

Example: predict mark for Statistical Machine Learning (SML) from mark for Intro ML (IML)



* synthetic data :)

Regression as a probabilistic model



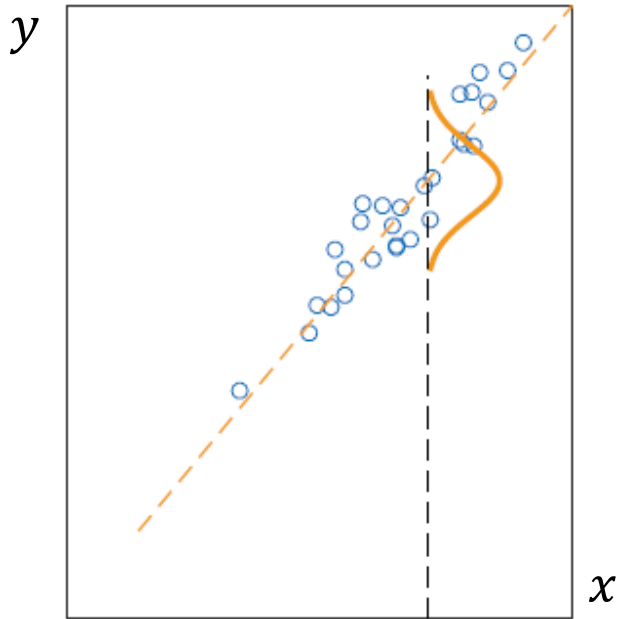
- Assume a **probabilistic model**:
 $y = \mathbf{x}'\mathbf{w} + \varepsilon$
 - * Here $\mathbf{x}$, y and ε are r.v.'s
 - * Variable ε encodes noise
- Next, assume Gaussian noise (indep. of $\mathbf{x}$): $\varepsilon \sim \mathcal{N}(0, \sigma^2)$
 thus: $y \sim \mathcal{N}(\mathbf{x}'\mathbf{w}, \sigma^2)$

- Recall that $\mathcal{N}(z; \mu, \sigma^2) \equiv \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(z-\mu)^2}{2\sigma^2}\right)$
- Therefore

$$p_{\mathbf{w}, \sigma^2}(y|\mathbf{x}) = \mathcal{N}(y; \mathbf{x}'\mathbf{w}, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mathbf{x}'\mathbf{w})^2}{2\sigma^2}\right)$$

this is a
squared
error!

Parametric probabilistic model



- Using simplified notation, **discriminative model** is:

$$p(y|x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - x'w)^2}{2\sigma^2}\right)$$

- Unknown parameters: w, σ^2

- Given observed data $\{(x_1, y_1), \dots, (x_n, y_n)\}$, we want to find parameter values that “best” explain the data
- Maximum-likelihood estimation**: choose parameter values that maximise the probability of observed data

Maximum likelihood estimation

- Assuming independence of data points, the probability of data is

$$p(y_1, \dots, y_n | x_1, \dots, x_n) = \prod_{i=1}^n p(y_i | x_i)$$

- For $p(y_i | x_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - x_i'w)^2}{2\sigma^2}\right)$
- “Log trick”: Instead of maximising this quantity, we can maximise its logarithm

$$\sum_{i=1}^n \log p(y_i | x_i) = -\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - x_i'w)^2 + C$$

here C doesn't depend on w (it's a constant)

the sum of squared errors!

- Under this model, maximising log-likelihood as a function of w is equivalent to minimising the sum of squared errors

Method of least squares

- Training data: $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$. Note bold face in $\mathbf{x}_i$
- For convenience, place instances in rows (so attributes go in columns), representing training data as an $n \times m$ matrix $\mathbf{X}$, and n vector $\mathbf{y}$
- Probabilistic model/decision rule assumes $\mathbf{y} \approx \mathbf{X}\mathbf{w}$
- To find $\mathbf{w}$, minimise the sum of squared errors

$$\begin{aligned}
 L &= \sum_{i=1}^n (y_i - \mathbf{x}_i' \mathbf{w})^2 \\
 &= \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \\
 &= (\mathbf{y} - \mathbf{X}\mathbf{w})'(\mathbf{y} - \mathbf{X}\mathbf{w}) && \text{since } \|\mathbf{u}\|^2 = \mathbf{u}'\mathbf{u} \\
 &= \mathbf{y}'\mathbf{y} - 2\mathbf{y}'\mathbf{X}\mathbf{w} + \mathbf{w}'\mathbf{X}'\mathbf{X}\mathbf{w}
 \end{aligned}$$

Method of least squares

- To find $\mathbf{w}$, minimise the sum of squared errors

$$L = \mathbf{y}'\mathbf{y} - 2\mathbf{y}'\mathbf{X}\mathbf{w} + \mathbf{w}'\mathbf{X}'\mathbf{X}\mathbf{w}$$

- Setting gradient to zero

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \dots, \frac{\partial L}{\partial w_m} \right]' = -2\mathbf{X}'\mathbf{y} + 2\mathbf{X}'\mathbf{X}\mathbf{w} = \mathbf{0}$$

- Solving for $\mathbf{w}$ yields

$$\hat{\mathbf{w}} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y}$$

Analytic solution:

- Write gradient
- Set to zero
- Solve for model

- * This system of equations called the **normal equations**
- * System is well defined only if the inverse exists



Bayesian derivation?

- Later in the semester: return of linear regression
- Fully Bayesian, with a posterior:
 - * Bayesian linear regression
- Bayesian (MAP) point estimate of weight vector:
 - * Adds a penalty term to sum of squared losses
 - * Equivalent to L_2 “regularisation” to be covered next week
 - * Called: ridge regression

Mini Summary

- Linear regression
 - * Simple, effective, “interpretable”, basis for many approaches
 - * Probabilistic frequentist derivation
 - * Solution by normal equationsLater in semester: Bayesian approaches

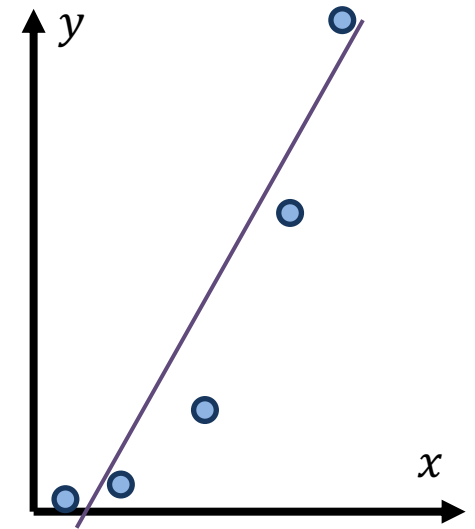
Next: Basis expansion for non-linear regression

Basis Expansion

Extending the utility of models via
data transformation

Basis expansion for linear regression

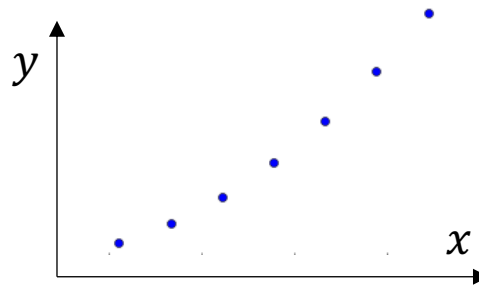
- Real data is likely to be non-linear
- What if we still wanted to use a linear regression?
 - * Simple, easy to understand, computationally efficient, etc.
- How to marry non-linear data to a linear method?



If you can't beat'em, join'em

Transform the data

- The trick is to **transform the data**: Map data into another features space, s.t. data is linear in that space
- Denote this transformation $\phi: \mathbb{R}^m \rightarrow \mathbb{R}^k$. If $\mathbf{x}$ is the original set of features, $\phi(\mathbf{x})$ denotes new feature set
- Example: suppose there is just one feature x , and the data is scattered around a parabola rather than a straight line

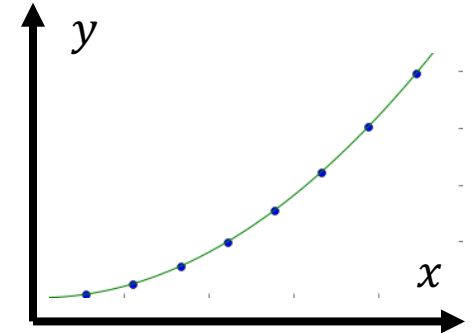


Example: Polynomial regression

- Define

- * $\varphi_1(x) = x$

- * $\varphi_2(x) = x^2$



- Next, apply linear regression to φ_1, φ_2

$$y = w_0 + w_1\varphi_1(x) + w_2\varphi_2(x) = w_0 + w_1x + w_2x^2$$

and here you have **quadratic regression**

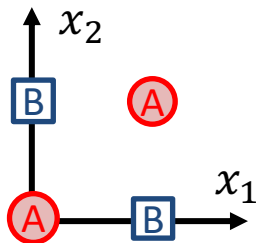
- More generally, obtain **polynomial regression** if the new set of attributes are powers of x

Example: linear classification

- Example binary classification problem: Dataset not **linearly separable**
- Define transformation as

$$\varphi_i(\mathbf{x}) = \|\mathbf{x} - \mathbf{z}_i\|, \text{ where } \mathbf{z}_i \text{ some pre-defined constants}$$

- Choose $\mathbf{z}_1 = [0,0]'$, $\mathbf{z}_2 = [0,1]'$, $\mathbf{z}_3 = [1,0]'$, $\mathbf{z}_4 = [1,1]'$



there exist weights that make new data separable, e.g.:

x_1	x_2	y
0	0	Class A
0	1	Class B
1	0	Class B
1	1	Class A

w_1	w_2	w_3	w_4
1	0	0	1

φ_1	φ_2	φ_3	φ_4
0	1	1	$\sqrt{2}$
1	0	$\sqrt{2}$	1
1	$\sqrt{2}$	0	1
$\sqrt{2}$	1	1	0

The transformed data is linearly separable!

$\varphi'w$	y
$\sqrt{2}$	Class A
2	Class B
2	Class B
$\sqrt{2}$	Class A

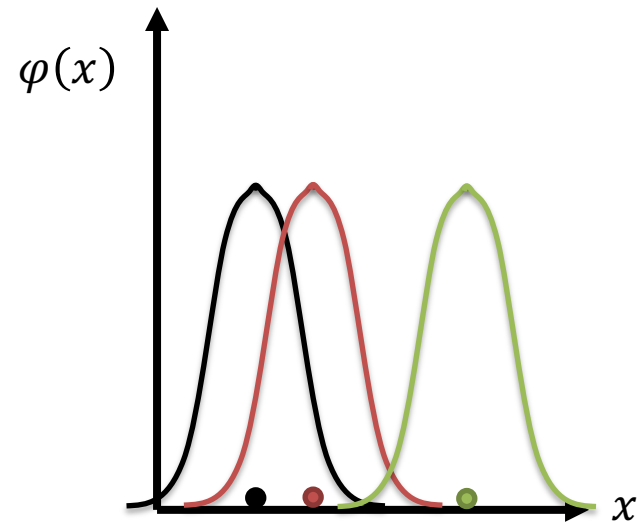
Radial basis functions

- Previous example: motivated by approximation theory where sums of RBFs approx. functions
- A **radial basis function** is a function of the form $\varphi(\mathbf{x}) = \psi(\|\mathbf{x} - \mathbf{z}\|)$, where $\mathbf{z}$ is a constant

- Examples:

- $\varphi(\mathbf{x}) = \|\mathbf{x} - \mathbf{z}\|$

- $\varphi(\mathbf{x}) = \exp\left(-\frac{1}{\sigma}\|\mathbf{x} - \mathbf{z}\|^2\right)$



Challenges of basis expansion

- Basis expansion can significantly increase the utility of methods, especially, linear methods
- In the above examples, one limitation is that the transformation needs to be defined beforehand
 - * Need to choose the size of the new feature set
 - * If using RBFs, need to choose $\mathbf{z}_i$
- Regarding $\mathbf{z}_i$, one can choose uniformly spaced points, or cluster training data and use cluster centroids
- Another popular idea is to use training data $\mathbf{z}_i \equiv \mathbf{x}_i$
 - * E.g., $\varphi_i(\mathbf{x}) = \psi(\|\mathbf{x} - \mathbf{x}_i\|)$
 - * However, for large datasets, this results in a large number of features $\rightarrow$ computational hurdle



Further directions

- There are several avenues for taking the idea of basis expansion to the next level
 - * Will be covered later in this subject
- One idea is to *learn* the transformation φ from data
 - * E.g., Artificial Neural Networks
- Another powerful extension is the use of the **kernel trick**
 - * “Kernelised” methods, e.g., kernelised perceptron
- Finally, in **sparse kernel machines**, training depends only on a few data points
 - * E.g., SVM

Mini Summary

- Basis expansion
 - * Extending model expressiveness via data transformation
 - * Examples for linear and logistic regression

Next time:

First/second-order iteration optimisation;

Logistic regression - linear probabilistic model for classification.